AlerGica

DOCUMENTAÇÃO TÉCNICA



Documentação Técnica do Sistema

Referência para desenvolvimento, implantação e manutenção do AlerGica — ambiente, configuração, dicionário de dados, APIs, serviços e componentes.

VERSÃO	ATUALIZADO EM	PÚBLICO	CLASSIFICAÇÃO
1.11.0	12 de novembro de 2026	Desenvolvedores e equipe de operação	Confidencial

Sumário

1. Sobre este documento

2. Pilha tecnológica

2.1 Frontend

2.2 Backend e infraestrutura

3. Estrutura do repositório

4. Ambiente de desenvolvimento

4.1 Pré-requisitos

4.2 Instalação

4.3 Scripts npm

5. Configuração

5.1 Variáveis do frontend (`.env` na raiz)

5.2 Variáveis da API FastAPI (`backend/.env`)

5.3 Segredos das Edge Functions (painel do Supabase)

6. Banco de dados

6.1 Convenções

6.2 Dicionário de dados — dados do usuário

6.3 Dicionário de dados — catálogo

6.4 Funções e gatilhos

6.5 Histórico de migrações

6.5.1 Correção do banco em produção (12/11/2026)

6.6 Como aplicar uma mudança de esquema

7. Camada de serviços do frontend

7.1 `services/medications.ts`

7.2 Regra de classificação — `medToRisk`

7.2.1 Regra por usuário — `lib/medicationRisk.ts`

7.3 Algoritmo de alternativas seguras

7.4 `services/auth.ts`

7.5 `services/profile.ts` e `lib/names.ts`

7.5.1 `services/privacy.ts` (LGPD)

7.5.2 `services/labelAnalysis.ts` (módulo Produtos)

7.5.3 `services/adminMessages.ts` (administrador)

7.5.4 Bulas — `services/bulas.ts` e `scripts/sync-bulas.mjs`

7.6 `services/studies.ts`

7.7 `services/api.ts`

8. APIs de servidor

8.1 Edge Function `fetch-aplv-news`

8.2 Edge Function `create-admin-user`

8.3 Edge Function `delete-account`

8.4 API FastAPI

8.5 Webhook n8n

9. Componentes de interface

10. Design system

10.1 Tokens de cor (tema claro, HSL)

10.2 Tipografia, sombra e movimento

11. Testes e qualidade

12. Implantação

12.1 Frontend (Vercel)

12.2 Banco e funções (Supabase)

12.3 API FastAPI

12.4 Checklist de publicação

13. Manutenção da documentação

14. Histórico de revisões

Documentação Técnica do Sistema

1. Sobre este documento

Este documento é a referência técnica do AlerGica: como montar o ambiente, configurar, executar, implantar e evoluir o sistema. Ele complementa o **Documento de Arquitetura** (visão estrutural e decisões) e o **Manual do Usuário** (uso do produto).

Documento	Arquivo-fonte	PDF
Arquitetura do Sistema	`docs/01-arquitetura.md`	`docs/pdf/01-arquitetura.pdf`
Documentação Técnica (este)	`docs/02-documentacao-tecnica.md`	`docs/pdf/02-documentacao-tecnica.pdf`
Manual do Usuário	`docs/03-manual-do-usuario.md`	`docs/pdf/03-manual-do-usuario.pdf`

2. Pilha tecnológica

2.1 Frontend

Biblioteca	Versão	Uso
React	18.3	Interface
TypeScript	5.8	Tipagem estática
Vite (+ plugin SWC)	5.4	Servidor de desenvolvimento e build
React Router DOM	6.30	Rotas da SPA
TanStack React Query	5.83	Cache de dados (provedor configurado na raiz)
Tailwind CSS	3.4	Estilos utilitários e tokens de tema
shadcn/ui + Radix UI	—	Componentes acessíveis (diálogos, drawer, carrossel, toasts...)
Framer Motion	12	Animações do menu inferior
Zod	3.25	Validação de formulários
lucide-react	0.462	Ícones
next-themes	0.3	Alternância tema claro/escuro
embla-carousel	8.6	Carrossel de conteúdo educativo
@supabase/supabase-js	2.91	Cliente de autenticação e dados
Vitest + Testing Library	3.2 / 16	Testes

2.2 Backend e infraestrutura

Componente	Tecnologia
Banco de dados	PostgreSQL (Supabase)
Autenticação	Supabase Auth
API REST	PostgREST (Supabase)
Funções serverless	Supabase Edge Functions (Deno)
API própria	Python 3 · FastAPI · Uvicorn · httpx · supabase-py · python-dotenv · python-jose
Automação de IA	n8n (webhook)
Hospedagem do frontend	Vercel

3. Estrutura do repositório

```
alergica/
├── backend/                  API FastAPI (Python)
│   ├── app/
│   │   ├── main.py           Aplicação, CORS e registro de routers
│   │   ├── db.py             Cliente Supabase autenticado com o JWT do usuário
│   │   └── routers/          medications.py · news.py
│   └── requirements.txt
├── docs/                     Esta documentação (Markdown + PDFs)
│   ├── assets/doc.css        Estilo dos PDFs
│   └── pdf/                  PDFs gerados
├── public/                   Arquivos estáticos (favicon, mascote, robots.txt)
├── scripts/
│   └── build-docs.mjs        Gera os PDFs da documentação
├── src/
│   ├── assets/               Imagens (mascote)
│   ├── components/           Componentes de tela
│   │   └── ui/               Componentes base shadcn/ui
│   ├── contexts/             AuthContext
│   ├── hooks/                use-mobile, use-toast
│   ├── integrations/supabase Cliente e tipos gerados do banco
│   ├── lib/utils.ts          Utilitário cn() para classes
│   ├── pages/                Telas (uma por rota)
│   ├── services/             Acesso a dados e regras de negócio
│   ├── test/                 Configuração e testes Vitest
│   ├── App.tsx               Provedores e rotas
│   ├── index.css             Tokens de design (claro/escuro) e classes utilitárias
│   └── main.tsx              Ponto de entrada
├── supabase/
│   ├── config.toml           Identificação do projeto Supabase
│   ├── functions/            Edge Functions (Deno)
│   └── migrations/           Migrações SQL versionadas
├── serve.mjs                 Servidor estático local (porta 3000)
├── screenshot.mjs            Captura de tela com Puppeteer
├── vercel.json               Reescrita de rotas da SPA
└── vite.config.ts            Porta 8080, alias @ e proxy /api
```

4. Ambiente de desenvolvimento

4.1 Pré-requisitos

- Node.js 18 ou superior e npm
- Python 3.11+ (somente para a API FastAPI)
- Supabase CLI (para aplicar migrações e publicar Edge Functions)
- Google Chrome ou Microsoft Edge (para gerar os PDFs da documentação)

4.2 Instalação

```
git clone <URL_DO_REPOSITORIO> alergica
cd alergica
npm install
cp .env.example .env      # crie o arquivo com as variáveis da seção 5
npm run dev               # http://localhost:8080
```

API FastAPI (opcional):

```
cd backend
python -m venv .venv
.venv\Scripts\activate          # Windows  (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt
uvicorn app.main:app --reload    # http://localhost:8000  ·  documentação em /docs
```

4.3 Scripts npm

Comando	O que faz
`npm run dev`	Servidor de desenvolvimento Vite em `http://localhost:8080` com recarga automática
`npm run build`	Build de produção em `dist/`
`npm run build:dev`	Build em modo desenvolvimento
`npm run preview`	Serve o build de produção pelo Vite
`npm run lint`	ESLint em todo o projeto
`npm run test`	Executa os testes uma vez (Vitest)
`npm run test:watch`	Testes em modo observação
`npm run docs:pdf`	Gera os PDFs da documentação em `docs/pdf/`

Comando	O que faz
`npm run bulas:sync -- …`	Sincroniza PDFs de bula com o Storage (ver 7.5.4). Exige `SUPABASE_SERVICE_ROLE_KEY` no ambiente
`node serve.mjs`	Serve `dist/` em `http://localhost:3000` (usado para capturas de tela)

5. Configuração

5.1 Variáveis do frontend (`.env` na raiz)

Variável	Obrigatória	Descrição
`VITE_SUPABASE_URL`	Sim	URL do projeto Supabase
`VITE_SUPABASE_ANON_KEY`	Sim	Chave pública (*anon*) do Supabase
`VITE_N8N_WEBHOOK_URL`	Sim, para IA	Webhook do n8n usado no onboarding e no scanner de produtos

> **Atenção:** variáveis com prefixo `VITE_` são embutidas no JavaScript entregue ao navegador. Nunca coloque nelas chaves secretas (como a *service role key*).

5.2 Variáveis da API FastAPI (`backend/.env`)

Variável	Descrição
`SUPABASE_URL`	URL do projeto Supabase
`SUPABASE_ANON_KEY`	Chave pública; as consultas usam o JWT do usuário, então a RLS se aplica
`NEWS_API_KEY`	Chave da NewsAPI para o endpoint `/news`

CORS liberado para `http://localhost:8080` e `http://localhost:3000`. Para produção, inclua o domínio público em `backend/app/main.py`.

5.3 Segredos das Edge Functions (painel do Supabase)

Segredo	Função
`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`	Fornecidos automaticamente pelo Supabase
`NEWS_API_KEY`	`fetch-aplv-news`

6. Banco de dados

6.1 Convenções

- Chaves primárias `uuid` com `gen_random_uuid()`.
- Datas em `timestamptz`; `created_at` com `now()` e `updated_at` mantido pelo gatilho `update_updated_at_column`.
- Toda tabela do esquema `public` com **RLS ativada**.
- Dados do usuário referenciam `auth.users(id)` pela coluna `user_id`.
- Uma migração SQL por mudança em `supabase/migrations/AAAAMMDDHHMMSS_descricao.sql`.

> **Atenção:** o esquema de produção da tabela `medications` (colunas em português) e as tabelas `products`, `restaurants` e `solicitado_inclusao` não estão nas migrações. O dicionário abaixo descreve o **banco em produção**, conforme `src/integrations/supabase/types.ts`, e as tabelas criadas pelas migrações. Ver item 1 da seção 13.2 "Itens em aberto" do documento de arquitetura.

6.2 Dicionário de dados — dados do usuário

`profiles` — **perfil do usuário**

Coluna	Tipo	Descrição
`id`	uuid PK	Identificador
`user_id`	uuid único → `auth.users`	Dono do perfil
`full_name`	text	Nome completo
`profile_type`	text	`alergica`, `mae`, `pai`, `farmaceutico` ou `medico`
`birth_date`	date	Data de nascimento (obrigatória no cadastro; usada para exigir 18+)
`gender`	text	`masculino`, `feminino`, `nao_binario` ou nulo (opcional)
`city` / `state`	text	Cidade e UF (opcionais)
`profession`	text	Profissão (opcional)
`phone`	text	Telefone
`food_allergies`	text[] (padrão `{}`)	Alergias alimentares marcadas — ids de `lib/allergens.ts` (`leite`, `ovo`, `soja`, `gluten`, `amendoim`, `castanhas`, `peixe`, `crustaceos`, `gergelim`, `milho`). Índice GIN

Coluna	Tipo	Descrição
`drug_allergies`	text[] (padrão `{}`)	Classes de medicamento marcadas (`penicilinas`, `cefalosporinas`, `sulfas`, `aines`, `aas`, `dipirona`, `paracetamol`, `macrolideos`, `quinolonas`, `anticonvulsivantes`). Índice GIN
`allergy_info`	text	Campo "Outros" (texto livre; também pode vir da IA no onboarding). Sem as colunas acima, guarda tudo como texto (reserva)
`observation`	text	Observações de saúde
`child_name`	text	Nome da criança (legado; substituído por `babies`)
`avatar_url`	text	Foto (não usada na interface atual)
`created_at` / `updated_at`	timestamptz	Auditoria

RLS: o usuário lê, cria e atualiza apenas o próprio perfil. Criado automaticamente no cadastro pelo gatilho `on_auth_user_created`.

`babies` — crianças vinculadas ao usuário

Coluna	Tipo	Descrição
`id`	uuid PK	Identificador
`user_id`	uuid	Responsável
`name`	text	Nome do bebê (obrigatório)
`birth_date`	date	Data de nascimento (opcional; usada para calcular a idade)
`created_at` / `updated_at`	timestamptz	Auditoria

RLS: CRUD completo apenas pelo dono.

`user_roles` — papéis de acesso

Coluna	Tipo	Descrição
`id`	uuid PK	Identificador
`user_id`	uuid → `auth.users`	Usuário
`role`	enum `app_role`	`admin`, `moderator` ou `user` (único por usuário + papel)

RLS: o usuário vê os próprios papéis; somente `admin` gerencia.

`user_onboarding` — controle da tela de boas-vindas

Coluna	Tipo	Descrição
`id`	uuid PK	Identificador
`user_id`	uuid único	Usuário que concluiu
`completed_at`	timestamptz	Quando concluiu

A ausência de linha faz o painel exibir o onboarding.

`user_consents` — histórico de consentimentos (LGPD)

Coluna	Tipo	Descrição
`id`	uuid PK	Identificador
`user_id`	uuid → `auth.users` (`ON DELETE CASCADE`)	Titular
`consent_type`	text	`termos_uso` , `politica_privacidade` ou `dados_saude`
`version`	text	Versão do documento aceito (`LEGAL.termsVersion` / `LEGAL.privacyVersion`)
`granted`	boolean	Aceito ou recusado
`created_at`	timestamptz	Momento do aceite

RLS: o titular lê e insere os próprios registros; `admin` lê todos; não há UPDATE nem DELETE pelo usuário (histórico só de inclusão). O registro principal também fica em `auth.users.raw_user_meta_data.lgpd_consents` , gravado no `signUp` — assim a prova do consentimento existe mesmo antes da migração ser aplicada.

`suggestions` e `feedback` — mensagens dos usuários

Coluna	Tipo	Descrição
`id`	uuid PK	Identificador
`user_id`	uuid	Autor
`content`	text	Mensagem. Pedidos de suporte vão para `suggestions` com o prefixo `[Suporte]`
`created_at`	timestamptz	Data de envio
`status`	text (padrão `novo`)	`novo` , `em_andamento` ou `resolvido` — definido pelo administrador
`status_updated_at`	timestamptz	Última mudança de status

RLS: o autor insere e lê as próprias mensagens; `admin` lê todas e altera o status. A equipe acompanha pela página **Mensagens** (`/admin/mensagens`).

`solicitado_inclusao` — pedidos de inclusão no catálogo

Coluna	Tipo	Descrição
`id`	uuid PK	Identificador
`user_id`	uuid	Solicitante
`tipo`	text	Medicamento, produto ou restaurante
`nome`	text	Nome do item
`composicao`	text	Composição informada
`imagem_url`	text	Foto do rótulo/bula no Storage
`status`	text	Situação da análise
`created_at`	timestamptz	Data

Tabela existente no banco, sem tela na versão atual.

6.3 Dicionário de dados — catálogo

`medications` — medicamentos (esquema em produção)

Coluna	Tipo	Descrição
`id`	uuid PK	Identificador
`nome_principal`	text	Nome exibido (comercial ou genérico)
`nome_alternativo`	text	Fabricante / laboratório
`nome_completo`	text	Nome completo do registro
`composicao`	text	Princípios ativos + `. Excipientes:` + lista de excipientes
`tem_risco_aplv`	boolean	`true` contém leite · `false` não contém · `null` indefinido
`nivel_alerta`	text	Rótulo de alerta (`VERDE` , `AMARELO` , `VERMELHO` ou texto equivalente)
`avisos`	jsonb	Avisos da análise (lista de textos)
`termos_encontrados`	jsonb	Termos lácteos encontrados na bula
`detalhes_criticos`	jsonb	Pontos críticos da análise por IA
`detalhes_atencao`	jsonb	Pontos de atenção da análise por IA
`arquivo_url`	text	URL do PDF da bula original

Coluna	Tipo	Descrição
`data_extracao`	text	Data da análise da bula
`access_count`	integer	Contador de visualizações (ordena "Mais Procurados")
`bula_path`	text	Cópia da bula no Storage (`bulas/<id>/<sha256>.pdf`); o app prefere este campo a `arquivo_url`
`bula_sha256`	text	Hash do PDF guardado — detecta mudança de conteúdo
`bula_atualizada_em` / `bula_origem`	timestamptz / text	Quando e por onde a versão entrou (`admin` , `script` , `original`)
`bula_reanalisar`	boolean (padrão `false`)	Bula trocada e análise ainda não conferida → o app mostra **Atenção**
`created_at` / `updated_at`	timestamptz	Auditoria

RLS: leitura por qualquer usuário autenticado; escrita somente `admin` .

`bula_versoes` — histórico de bulas

Coluna	Tipo	Descrição
`id`	uuid PK	Identificador
`medication_id`	uuid	Remédio
`bula_path` / `bula_sha256`	text	Arquivo e hash da versão
`origem`	text	`admin` , `script` ou `original`
`enviado_por`	uuid	Administrador que enviou (nulo no script)
`created_at`	timestamptz	Data

RLS: leitura só `admin` ; gravação pelas funções abaixo ou pelo script (chave de serviço). **Storage:** bucket `bulas` público para leitura (bulas são documentos públicos), PDF até 20 MB; envio, troca e exclusão só `admin` .

`products` e `restaurants` — módulos futuros

Mesma estrutura-base de `medications` (`nome_principal` , `nome_alternativo` , `nome_completo` , `nivel_alerta` , `tem_risco_aplv` / `avisos` , `access_count`). `products` tem `composicao` ; `restaurants` tem `descricao` e `endereco` . Ainda sem uso na interface.

Modelo normalizado (migração `create_medication_catalog`)

Tabela	Colunas principais	Finalidade
`allergen_groups`	`name` , `code` (`MILK` , `LACTOSE` , `GLUTEN` , `SOY` , `EGG`), `description`	Categorias de alergia

Tabela	Colunas principais	Finalidade
`ingredients`	`name` (único, sem diferenciar caixa), `common_names[]`, `cas_number`, `is_allergen`, `allergen_group_id`	Substâncias
`medication_ingredients`	`medication_id`, `ingredient_id`, `is_excipient`, `quantity`, `notes`	Composição normalizada

A migração `seed_medications` popula 5 grupos, 34 ingredientes e 25 medicamentos de exemplo.

`news_cache` — cache de notícias

Coluna	Tipo	Descrição
`title`, `description`, `url` (único), `source`	text	Dados da notícia
`published_at`	timestamptz	Data de publicação
`created_at` / `updated_at`	timestamptz	Usado para expirar o cache em 24 h

6.4 Funções e gatilhos

Objeto	Tipo	Descrição
`has_role(_user_id uuid, _role app_role) → boolean`	Função `SECURITY DEFINER`	Verifica se o usuário tem o papel. Usada nas políticas e pelo frontend (`isAdmin`).
`admin_list_messages() → table`	Função `SECURITY DEFINER`	Só `admin` (senão erro `42501`). Une `suggestions` e `feedback` (até 1000, mais recentes primeiro) com `kind` (`suporte` pelo prefixo `[Suporte]`, `sugestao`, `feedback`), conteúdo sem o prefixo, status, nome do perfil e e-mail de `auth.users`. `EXECUTE` só para `authenticated`
`admin_set_bula(_med_id, _path, _sha256, _origem) → text`	Função `SECURITY DEFINER`	Só `admin`. Hash igual ao atual → `igual` (nada muda); senão grava a versão, registra em `bula_versoes` e liga `bula_reanalisar`. Retorna `primeira` ou `atualizada`
`admin_mark_bula_analisada(_med_id) → void`	Função `SECURITY DEFINER`	Só `admin`. Desliga `bula_reanalisar` depois que a análise foi refeita/conferida (o fluxo de análise também pode gravar `false` diretamente)
`increment_medication_access(med_id uuid) → void`	Função `SECURITY DEFINER`	Soma 1 em `access_count`. `EXECUTE` liberado para `authenticated`.
`handle_new_user()`	Função de gatilho	Cria `profiles` (com `full_name` dos metadados) e `user_roles('user')`

Objeto	Tipo	Descrição
`on_auth_user_created`	Gatilho em `auth.users`	Executa `handle_new_user` após cada cadastro
`update_updated_at_column()`	Função de gatilho	Atualiza `updated_at` em `profiles`, `babies`, `ingredients`, `medications`, `news_cache`

6.5 Histórico de migrações

Arquivo	Conteúdo
`20260125174611_…sql`	`profiles`, `app_role`, `user_roles`, `has_role`, `user_onboarding`, gatilhos de cadastro
`20260127005620_…sql`	`babies`, `suggestions`, `feedback`
`20260306000000_add_profile_extra_fields.sql`	`phone`, `allergy_info`, `observation` em `profiles`
`20260306000001_create_medication_catalog.sql`	Catálogo normalizado e RLS
`20260306000002_seed_medications.sql`	Dados de exemplo do catálogo
`20260306000003_add_access_count_and_profile_type.sql`	`access_count` e `profile_type`
`20260306000004_create_news_cache.sql`	`news_cache`
`20260306000005_increment_medication_access_fn.sql`	Função de contador de acessos
`20260308000000_add_profile_demographics.sql`	`birth_date`, `gender`, `city`, `state`, `profession`
`20261112000000_lgpd_consents.sql`	`user_consents` (histórico de consentimentos) e políticas RLS
`20261112000001_profile_allergy_lists.sql`	`food_allergies`, `drug_allergies` em `profiles` (+ índices GIN). Em produção: `supabase/manual/2026-11-12_alergias_por_tipo.sql`
`20261112000002_admin_messages.sql`	`status` / `status_updated_at` em `suggestions` e `feedback`, políticas de leitura e atualização para `admin`, função `admin_list_messages`. Em produção: `supabase/manual/2026-11-12_mensagens_admin.sql`

Arquivo	Conteúdo
`20261112000003_bulas_storage.sql`	Colunas `bula_*` em `medications`, tabela `bula_versoes`, bucket `bulas` e políticas, funções `admin_set_bula` e `admin_mark_bula_analisada`. Em produção: `supabase/manual/2026-11-12_bulas_storage.sql`

6.5.1 Correção do banco em produção (12/11/2026)

Verificado pela API: o banco em uso **não tinha** as colunas `phone`, `allergy_info`, `observation`, `birth_date`, `gender`, `city`, `state` e `profession` em `profiles`, a tabela `user_consents` nem a função `increment_medication_access`. Efeito: salvar o perfil falhava, os dados complementares do cadastro e as alergias do onboarding eram descartados sem aviso, e o contador de "Mais Procurados" não mudava.

Correção: `supabase/manual/2026-11-12_corrigir_perfil_lgpd_contador.sql` — idempotente, só adiciona o que falta. Executar no **SQL Editor** do Supabase. **Não usar** `supabase db push` **neste banco**: as migrações antigas tentariam recriar `medications`, que em produção tem outro formato.

Desde esta versão, o cadastro grava primeiro `full_name` e `profile_type` e depois os dados complementares, e os erros de gravação são registrados no console.

6.6 Como aplicar uma mudança de esquema

1. Crie `supabase/migrations/AAAAMMDDHHMMSS_descricao.sql` (use `IF NOT EXISTS` quando fizer sentido).
2. Habilite RLS e crie as políticas na mesma migração de qualquer tabela nova.
3. Aplique: `supabase db push`.
4. Regenere os tipos: `supabase gen types typescript --project-id <id> > src/integrations/supabase/types.ts`.
5. Atualize o dicionário de dados neste documento e gere os PDFs (`npm run docs:pdf`).

7. Camada de serviços do frontend

Regra do projeto: **componentes não chamam APIs diretamente** — usam as funções de `src/services/`.

7.1 `services/medications.ts`

Função	Retorno	Descrição
`getPopularMedications()`	`MedRow[]`	6 medicamentos com maior `access_count`
`getMedicationsPage(page, pageSize = MEDS_PAGE_SIZE)`	`{ items: MedRow[], total }`	Uma página do catálogo em ordem alfabética (desempate por `id`), com total via `count: "exact"`. Usada no filtro "Todos" do módulo Remédios
`getRecentMedications(limit = 10)`	`MedRow[]`	Lista em ordem alfabética (sem uso na interface atual)
`getMedicationsByRisk(risk, alergias, personalizado)`	`MedRow[]`	Lista completa de um nível para o perfil. Sem personalização: pré-filtra no banco por `tem_risco_aplv` em lotes de 1000 linhas. Com outras alergias: lê o catálogo inteiro (cache de 10 min) e filtra com `assessMedication`
`searchMedications(q, limit = 12)`	`MedRow[]`	Busca `ILIKE` em `nome_principal`, `nome_alternativo` e `nome_completo`. O painel chama com `limit = 300`; o Início exibe os 12 primeiros
`getMedication(id)`	`MedicationDetail`	Registro completo
`getSafeAlternatives(med, limit = 6, alergias?)`	`SafeAlternative[]`	Mesmos princípios ativos, sem leite, um por fabricante; com `alergias`, só aceita as que `assessMedication` classifica como **Seguro**
`recordMedicationView(id)`	`void`	Chama a RPC de contador; falhas são ignoradas
`medToRisk(temRisco, nivel, avisos)`	`"safe" | "caution" | "risk"`	Classificação do catálogo **para leite** (= `catalogMilkRisk`, ver 7.2). Para o usuário use `assessMedication`
`parseTextField(raw)`	`string[]`	Normaliza campos JSON/texto em lista de strings
`activeKeysOf(med)`	`string[]`	Chaves canônicas de princípios ativos

Tipos e constantes exportados: `MedRow` (campos de listagem, inclui `composicao`), `MedicationDetail` (registro completo), `SafeAlternative`, `MEDS_PAGE_SIZE` (30).

Paginação do módulo Remédios. Com o filtro "Todos" e sem busca, a paginação é feita no banco (`range`), buscando só os 30 itens da página. Nos filtros Seguro/Atenção/Risco a classificação final depende de `medToRisk` (que lê `avisos`), então a lista filtrada é carregada uma vez e paginada no navegador; trocar de página não faz nova consulta. Resultados de busca também são paginados no navegador. Trocar filtro ou busca volta para a página 1.

7.2 Regra de classificação — `medToRisk`

Avaliada na ordem abaixo; a primeira condição verdadeira decide.

Ordem	Condição	Resultado
1	`nivel_alerta` contém "crítico", "critico" ou "proibido"	`risk`
2	`avisos` contém "crítico", "critico", "proibido" ou 🔴	`risk`
3	`nivel_alerta` contém "atenção", "atencao" ou "cuidado"	`caution`
4	`avisos` contém "atenção", "atencao", "cuidado" ou 🟡	`caution`
5	`tem_risco_aplv = true`	`risk`
6	`tem_risco_aplv = false`	`safe`
7	`nivel_alerta` contém "seguro" ou "liberado"	`safe`
8	Nenhuma das anteriores	`caution` (padrão seguro)

7.2.1 Regra por usuário — `lib/medicationRisk.ts`

`assessMedication(med, alergias)` devolve `level` , `hits` (alergias encontradas), `ambiguous` (ingredientes de origem não informada), `cross` (reação cruzada), `unverified` (alergias sem composição para verificar), `milkLevel` / `milkChecked` . O pior resultado vence:

1. Leite no perfil (ou perfil vazio) → nível do catálogo (`catalogMilkRisk`).
2. Alergia alimentar na `composicao` , ou alergia a medicamento/termo próprio no nome, `nome_completo` ou `composicao` → `risk` .
3. Ingrediente ambíguo (`ambiguous` de cada grupo, com complementos que esclarecem a origem em `unless`), reação cruzada (`crossReactive`) ou composição ausente com alergias além de leite → `caution` .
4. Nada encontrado → `safe` .

`assessmentBadge(a, personalizado)` gera o texto do selo ("Seguro para o seu perfil", "Atenção (Verificar)", "Risco: leite e ovo"). Usado em `Dashboard` (listas e busca), `MedicationDetails` e `getSafeAlternatives` . Testes em `lib/medicationRisk.test.ts` .

7.3 Algoritmo de alternativas seguras

1. `activeKeysOf` extrai princípios ativos do `nome_principal` e do trecho da `composicao` antes de "Excipientes".
2. `normalizeName` remove acentos, ®/™, parênteses (inclusive aninhados) e dosagens (`500 mg` , `10 mg/ml` , `%` , `UI`).
3. A lista é separada por `+` , `;` , `,` ou " e "; cada item passa por `canonicalIngredient` , que remove prefixos de laboratório público (`furp-` , `lfm-`), sais, hidratos e formas farmacêuticas.
4. Itens únicos são ordenados e unidos por " + " → **chave canônica**.
5. A palavra mais longa (≥ 4 letras) de cada chave vira termo de busca, com vogais e "c" trocados por `_` (ignora acentos no `ILIKE`).
6. Consulta: `tem_risco_aplv = false` , id diferente do atual, termo em nome ou composição, ordenado por `access_count` , até 300 linhas.
7. No cliente: mantém só registros cuja chave canônica é idêntica; deduplica por fabricante normalizado (sem `Ltda` , `S.A.` , `EIRELI` , `ME` , `EPP`); para em 6.

7.4 `services/auth.ts`

Função	Descrição
`requestPasswordReset(email)`	`resetPasswordForEmail` com `redirectTo = <origem>/redefinir-senha`
`updatePassword(password)`	`auth.updateUser({ password })` na sessão de recuperação

> **Atenção:** cadastre `https://<dominio>/redefinir-senha` em Supabase → Authentication → URL Configuration → *Redirect URLs*, senão o link do e-mail é recusado.

7.5 `services/profile.ts` e `lib/names.ts`

Função	Descrição
`getProfileSummary(userId)`	`full_name` , `profile_type` e `gender` do perfil, para o menu lateral
`getAllergyInfo(userId)`	Texto livre `allergy_info` do perfil; usado pelo hook `useUserAllergies` (`hooks/use-user-allergies.ts` , React Query, chave `user-allergies` , invalidada ao salvar o perfil e no onboarding)
`shortName(nome)`	Dois primeiros nomes, mantendo partículas entre eles e corrigindo maiúsculas ("CAROLINE LIMA MARQUES" → "Caroline Lima"; "Maria de Souza Santos" → "Maria de Souza")
`initials(nome)`	Iniciais para o avatar, ignorando partículas
`nameFromEmail(email)`	Nome provisório a partir do e-mail, quando não há nome cadastrado

Função	Descrição
`displayName(nomePerfil, user)`	Nome curto na ordem perfil → cadastro → e-mail; usado no menu, no perfil e na saudação do painel
`firstName(nome)`	Primeiro nome, para saudações

`lib/theme.ts`: `applySavedTheme`, `setDarkTheme`, `isDarkTheme` e `onThemeChange` (evento `alergica:theme`). Tema salvo em `localStorage("theme")` e aplicado pela classe `dark` no `<html>`.

O menu usa o nome nesta ordem: `profiles.full_name` → `user_metadata.full_name` (cadastro) → e-mail.

`lib/allergens.ts` **— catálogo de alergias:**

Função / constante	Descrição
`ALLERGEN_GROUPS`	Grupos com `kind` (alimento/medicamento), `featured` (aparece nas listas), `aliases` (o que o usuário escreve), `terms` (o que é procurado), `ambiguous` e `crossReactive`. Além dos 20 das listas: corantes, látex, opioides, tetraciclinas, anestésicos locais, iodo (só via "Outros")
`FEATURED_FOOD` / `FEATURED_DRUG`	As 10 alimentares e as 10 classes de medicamentos das listas
`parseAllergyInfo(texto)`	Reconhece o "Outros": separa por vírgula, ponto e vírgula, barra, quebra de linha, "e"/"ou"; item com apelido → grupo; demais → termo próprio
`buildUserAllergies(sel)`	Junta listas marcadas e o reconhecido em "Outros"
`selectionFromRow(linha)`	Linha de `profiles` → seleção. Perfis antigos (só texto) têm os itens das listas marcados automaticamente
`selectionToText(sel)`	Texto de reserva quando o banco não tem as colunas novas (reconhecido de volta)
`effectiveAllergies(a)`	Sem alergias → só leite (APLV)
`findAllergens`, `findAmbiguous`, `foodOnly`, `joinLabels`, `shortLabel`	Busca em textos e auxiliares

`services/profile.ts`: `getAllergySelection(userId)` e `saveAllergySelection(userId, sel)` — grava `food_allergies`, `drug_allergies` e `allergy_info`; se o banco responder coluna inexistente (`42703` / `PGRST204`), grava tudo como texto em `allergy_info` e devolve `fallback: true`. Hook `useUserAllergies` (`hooks/use-user-allergies.ts`, React Query, chave `user-allergies`) entrega `selection`, `profile`, `effective`, `personalized` e `loading`; é invalidado ao salvar o perfil e no onboarding.

Componentes: `AllergyPicker` (listas + "Outros" com reconhecimento ao vivo; itens reconhecidos no texto aparecem marcados com "reconhecido em Outros"), usado no cadastro e no perfil; `AllergySummary` (alergias por tipo, no quadro "Saúde"). Testes em `lib/allergens.test.ts`.

7.5.1 `services/privacy.ts` (LGPD)

Função	Descrição
`buildSignupConsents()`	Monta os três consentimentos do cadastro (termos, política, dados de saúde) com versão e data
`logConsents(userId, consents)`	Grava o histórico em `user_consents` (falha não bloqueia o cadastro)
`getMyConsents()`	Lê `user_metadata.lgpd_consents` do usuário logado
`exportMyData()` / `downloadMyData()`	Junta conta, perfil, bebês, sugestões, feedback, onboarding e consentimentos e baixa como JSON (art. 18, II e V)
`deleteMyAccount()`	Chama a Edge Function `delete-account` e encerra a sessão (art. 18, VI)

Os dados do controlador, do encarregado e as versões dos documentos ficam em `src/config/legal.ts` (`LEGAL`). **Preencha os campos entre colchetes antes de publicar** e aumente a versão sempre que mudar o texto da Política ou dos Termos.

7.5.2 `services/labelAnalysis.ts` (módulo Produtos)

Função / constante	Descrição
`analyzeLabel(blob, alergias?, onProgress?)`	Tenta o n8n (envia também `allergies`, lista JSON com os nomes das alergias alimentares do perfil) (se `VITE_N8N_WEBHOOK_URL` existir; tempo limite 30 s; exige `safe` booleano). Se não estiver configurado ou falhar, faz a leitura local. Lança erro só se as duas falharem
`classifyLabelText(texto, alergias?)`	Regras sobre o texto do rótulo para as alergias do perfil (padrão: leite). Retorna `hits` agrupados por alergia. `contains` (termo fora de "pode conter"/"não contém"), `traces` (só em "pode conter", tolerando falhas de leitura como "pode conler") ou `inconclusive`. Ignora "zero/sem lactose" e palavras parecidas ("Deleite")
`MILK_TERMS`	leite, lactose, caseína, caseinato, lactoalbumina, lactoglobulina, soro de leite, whey, manteiga, creme de leite, queijo, requeijão, iogurte, lácteo(a)(s), gordura do leite, nata
`MIN_CONFIDENCE` (70)	Leitura abaixo disso (ou com menos de 40 letras) vira "foto pouco nítida", exceto quando encontrou leite
`estimateSkew(canvas)`	Descobre a inclinação do texto (-20° a +20°) por perfil de projeção numa miniatura de 500 px (~0,2 s)

Preparação da imagem (canvas): lado maior padronizado em 2000 px → tons de cinza com contraste esticado (ignora 1% dos extremos) → texto endireitado pelo ângulo estimado. Leitura em modo bloco único (PSM 6) e, se não encontrar leite, segunda leitura automática (PSM 3), ficando com a melhor. **Tolerância a erros de leitura:** termos de uma palavra com 5+ letras aceitam 1 letra trocada (`le1te`, `lelte`, `lactcse`), após corrigir confusões comuns (1→l, 0→o, 3→e, 5→s); o erro, quando houver, vai para o lado seguro (indicar leite).

Leitura local: `tesseract.js` (idioma `por`), importado sob demanda (chunk separado de ~19 KB). Na primeira vez o navegador baixa o motor (WebAssembly) e os dados do idioma pela CDN jsDelivr — alguns MB; nenhum dado do usuário é enviado. `vite.config.ts` inclui `tesseract.js` em `optimizeDeps` para evitar erro de importação dinâmica no modo de desenvolvimento (reinicie o `npm run dev` depois de instalar dependências).

Regra de segurança: a leitura local nunca resulta em "seguro". Testes em `labelAnalysis.test.ts` (12 casos, incluindo texto real lido de rótulos e alergias a ovo, amendoim e termos próprios).

Teste com rótulos reais (12/11/2026, Chrome, versão com preparação da imagem):

Foto	Rótulo diz	Resultado	Leitura
Barra de cereal (foto inclinada ~10°, amarelo/marrom)	leite em pó, contém lactose	contém (leite, lactose)	85% (antes: 40%, nada encontrado)
Nescau	soro de leite	contém	80%
Pão Panco	pode conter leite	contém (exagera, lado seguro)	71%
Toddy (foto pequena e borrada)	pode conter leite	contém (exagera, lado seguro)	58%
Pão Pullman	pode conter leite (o texto do Open Food Facts omitia)	pode conter traços	89%
Tapioca	sem leite	inconclusivo (nunca "seguro")	85%

1,3–2,4 s por foto; ~11 s na primeira análise (download do motor e do idioma).

7.5.3 `services/adminMessages.ts` (administrador)

Função / tipo	Descrição
`listAdminMessages()`	Chama a RPC `admin_list_messages`
`updateMessageStatus(msg, status)`	Atualiza `status` e `status_updated_at` na tabela de origem (`msg.source`)
`AdminSetupMissingError`	Função/colunas inexistentes no banco (script ainda não aplicado) — a página mostra a instrução
`AdminForbiddenError`	Usuário sem papel `admin`
`KIND_LABELS`, `STATUS_LABELS`	Textos de tipo e situação

`AuthContext` expõe `roleLoading`, verdadeiro até o papel do usuário atual ser checado — evita redirecionar o administrador antes da resposta de `has_role`.

Tornar alguém administrador (SQL Editor): `INSERT INTO public.user_roles (user_id, role) SELECT id, 'admin' FROM auth.users WHERE email = '…' ON CONFLICT DO NOTHING;`

7.5.4 Bulas — `services/bulas.ts` e `scripts/sync-bulas.mjs`

Função	Descrição
`bulaPublicUrl(path)`	URL pública do PDF no bucket `bulas`
`uploadBula(medId, file)`	Admin: confere se é PDF (`%PDF` , até 20 MB), calcula SHA-256 no navegador, envia `<id>/<sha>.pdf` e chama `admin_set_bula` . Retorna `primeira` , `atualizada` ou `igual`
`markBulaAnalyzed(medId)`	Admin: chama `admin_mark_bula_analisada`

As listas leem `bula_reanalisar` junto com o catálogo; se a coluna ainda não existir no banco, `medications.ts` repete a consulta sem ela (`withListSelect`). `assessMedication` trata `bula_reanalisar = true` como **Atenção** (`bulaPending`).

Script em lote (`npm run bulas:sync -- <opções>` , chave de serviço só no ambiente local, nunca no repositório):

Opção	Efeito
`--dir <pasta>`	Envia PDFs nomeados `<medication_id>.pdf` ; versão diferente da guardada liga `bula_reanalisar`
`--from-urls`	Para remédios sem cópia, baixa o `arquivo_url` (é a versão já analisada: não exige nova análise). Links expirados/bloqueados aparecem como falha
`--dry-run` , `--limit n` , `--delay ms`	Simulação, limite de itens, pausa entre downloads (padrão 1500 ms)

Ex.: `SUPABASE_SERVICE_ROLE_KEY=… npm run bulas:sync -- --from-urls --dry-run` . O fluxo de análise (n8n) deve gravar `bula_reanalisar = false` ao concluir uma nova análise.

7.6 `services/studies.ts`

Item	Detalhe
`getRecentStudies()`	Busca, une, ordena por data e traduz os estudos
Fonte	`https://www.ebi.ac.uk/europepmc/webservices/rest/search` (somente `SRC:MED` , sem cartas e comentários)
Categoria `formulas`	Título com "milk allergy" + fórmula/hidrolisado/aminoácidos, 2024 em diante, até 7
Categoria `medicamentos`	Título com "milk allergy" + medicamento/excipiente/vacina/inalador, 2019 em diante, até 5
Link	PubMed (`pmid`) → DOI → página Europe PMC

Item	Detalhe
Tradução	MyMemory `en → pt-BR`, tempo limite de 4 s, cache em `localStorage` (`alergica:study-title-pt`)

7.7 `services/api.ts`

`apiFetch<T>(path, options)` chama `/api{path}` com o cabeçalho `Authorization: Bearer <JWT>` da sessão atual e lança erro `API <status>: <corpo>` em respostas não-2xx. Preparado para a API FastAPI; ainda sem chamadas na interface.

8. APIs de servidor

8.1 Edge Function `fetch-aplv-news`

Item	Detalhe
Método	`POST` / `GET` (com pré-voo `OPTIONS` para CORS)
Resposta	`[{ title, description, url, publishedAt, source }]` — até 10 itens
Lógica	Devolve o cache (`news_cache`) se houver itens com menos de 24 h; senão consulta a NewsAPI em pt, en e fr, remove duplicados e "[Removed]", grava o cache (`upsert` por `url`)
Erros	Sempre devolve lista (vazia em caso de falha)

8.2 Edge Function `create-admin-user`

Item	Detalhe
Autenticação	`Authorization: Bearer <JWT de um usuário com papel admin>`
Corpo	`{ email, password, fullName?, makeAdmin? }`
Resposta	`200 { success, user: { id, email } }` · `400 { error }` · `401` sem login válido · `403` se não for `admin` · `500 { error }`
Lógica	Valida o JWT (`auth.getUser`) e o papel (`has_role`); então cria usuário confirmado com a chave de serviço e, com `makeAdmin` , grava papel `admin`

> **Nota:** a chave pública (*anon*) também é um JWT válido para a plataforma, por isso a função faz a própria checagem de usuário e papel em vez de confiar só no `verify_jwt` . O primeiro administrador deve ser criado pelo painel do Supabase (inserindo a linha em `user_roles`).

8.3 Edge Function `delete-account`

Item	Detalhe
Autenticação	`Authorization: Bearer <JWT do próprio usuário>` (o SDK envia automaticamente em `functions.invoke`)
Resposta	`200 { success: true }` · `401` sem login válido · `500 { success: false, error }`
Lógica	Apaga as linhas do usuário em `babies` , `suggestions` , `feedback` , `solicitado_inclusao` , `user_consents` , `user_onboarding` , `user_roles` e `profiles` (tabelas inexistentes são ignoradas) e depois exclui o usuário com `auth.admin.deleteUser`

Item	Detalhe
Publicação	`supabase functions deploy delete-account`

Arquivos no Storage (imagens de `solicitado_inclusao`) ainda não são apagados pela função.

8.4 API FastAPI

Todos os endpoints de medicamentos exigem `Authorization: Bearer <JWT do Supabase>`.

Método e rota	Parâmetros	Resposta
`GET /medications/search`	`q` (texto, opcional), `limit` (padrão 12)	Lista de medicamentos; sem `q`, os 6 mais acessados
`GET /medications/{medication_id}`	—	Medicamento com `medication_ingredients` → `ingredients` → `allergen_groups`; `404` se não existir
`POST /medications/{medication_id}/view`	—	`{ "ok": "true" }` (falha do contador é ignorada)
`GET /news`	—	Até 10 notícias; `500` se `NEWS_API_KEY` ausente

Documentação interativa em `http://localhost:8000/docs`.

8.5 Webhook n8n

Uso	Requisição	Resposta esperada
Onboarding — perfil de alergias	`POST` JSON `{ "allergies": "<texto livre>" }`	Texto com o perfil gerado (qualquer campo de texto retornado é aceito)
Scanner de produtos	`POST` multipart, campo `image` (JPEG)	`{ safe: boolean, product?: string, allergens?: string[], details?: string }`. Se `safe` não vier como booleano, ou a chamada falhar, o app usa a leitura local (seção 7.5.2) — nunca um resultado padrão.

9. Componentes de interface

Componente	Responsabilidade
`Sidebar`	Barra lateral (desktop) e conteúdo do drawer (celular): logo, tema, cartão do usuário (dois primeiros nomes, iniciais e, para farmacêutico/médico, a profissão no feminino ou masculino conforme `gender`), menu Início / Meu Perfil / Configurações (para `admin`, também **Mensagens**, com o número de mensagens novas), Sair e links legais
`ExpandableTabs`	Menu inferior fixo com os módulos Início, Remédios, Produtos, Restaurantes, Nutrição
`MedicationCard`	Card de medicamento com nome, fabricante e selo de risco
`RiskBadge`	Selo colorido do nível de risco
`PaginationBar`	Navegação entre páginas (Anterior, números com reticências, Próxima; no celular mostra "página / total")
`APLVInfoCarousel`	Carrossel com 10 fichas educativas com fonte e estudos recentes traduzidos
`OnboardingModal`	Boas-vindas e configuração do perfil de alergias com IA (pulada para profissionais de saúde)
`LegalLayout`	Layout das páginas públicas de Política de Privacidade e Termos de Uso
`ProductScanner`	Câmera, captura, envio ao n8n e exibição do resultado; estados `idle`, `camera`, `preview`, `analyzing`, `result` ou `error`; foto pela câmera ou da galeria (imagem até 10 MB); no resultado, a foto fica ao lado do veredito e abre ampliada em um `Dialog`
`BabyManager`	Lista, inclusão e remoção de bebês, com cálculo de idade
`ChatWidget`	Botão flutuante do mascote: Suporte, Sugestão e Feedback
`BulaViewer`	Diálogo com o PDF da bula guardada (`iframe`; no Android abre no leitor de PDF), com Baixar e Abrir em nova aba
`AdminMessagesPage`	Página `/admin/mensagens`: contadores por situação (também filtram), abas por tipo, busca por texto/nome/e-mail, troca de situação com atualização otimista, copiar e-mail e responder (`mailto:`); atualiza a cada 60 s
`ThemeToggle`	Botão de tema da barra lateral; usa `lib/theme.ts` e fica sincronizado com o seletor de Configurações
`PageHeader`	Cabeçalho padrão das páginas internas: menu (celular), voltar, título `font-display` e subtítulo
`NavLink`	Link de navegação com estado ativo
`ui/*`	Biblioteca shadcn/ui (botões, cards, drawer, diálogo, carrossel, skeleton, toasts...)

10. Design system

10.1 Tokens de cor (tema claro, HSL)

Token	Valor	Uso
`--primary`	`152 76% 65%`	Verde-menta da marca, botões e destaques
`--background`	`120 20% 98%`	Fundo da aplicação
`--foreground`	`160 25% 20%`	Texto principal
`--secondary`	`140 30% 94%`	Superfícies suaves
`--safe`	`152 70% 55%`	Nível Seguro
`--caution`	`40 90% 60%`	Nível Atenção
`--risk`	`0 65% 70%`	Nível Risco
`--radius`	`1rem` (lg `1.5rem`, xl `2rem`)	Cantos arredondados

Há um conjunto equivalente para o tema escuro (classe `.dark`). Use sempre os tokens (`bg-primary`, `text-risk-foreground`...), nunca cores fixas.

10.2 Tipografia, sombra e movimento

- Fontes: **Nunito** (400–800) no texto e **Fraunces** (600–700, classe `font-display`) em títulos de destaque — tela de login, documentos legais, 404.
- Utilitário `.grain`: textura de ruído SVG sobre superfícies com gradiente (painel da marca no login).
- Classe `.legal-doc`: tipografia de leitura dos documentos legais.
- Sombras em camadas, tingidas com a cor da marca (`--shadow-soft`, `--shadow-card`, `--shadow-hover`).
- Animações utilitárias: `animate-fade-in`, `animate-slide-in-right`, `animate-float`. Anime apenas `transform` e `opacity`; não use `transition-all`.
- Todo elemento clicável precisa de estados `hover`, `focus-visible` e `active`.

11. Testes e qualidade

Item	Estado
Executor	Vitest com ambiente jsdom e Testing Library (`src/test/setup.ts`)
Cobertura atual	Teste de exemplo e `src/lib/names.test.ts` (formatação de nomes)
Prioridade 1	Testes unitários de `medToRisk` , `parseTextField` , `activeKeysOf` , `getSafeAlternatives` (com cliente simulado)
Prioridade 2	Testes de componentes: `AuthPage` (validações), `MedicationDetails` (destaque de excipientes)
Prioridade 3	Testes ponta a ponta (Playwright) de cadastro, busca e detalhe
Lint	ESLint 9 com regras de React Hooks (`npm run lint`)

12. Implantação

12.1 Frontend (Vercel)

1. Conecte o repositório ao Vercel (framework: Vite).
2. Comando de build `npm run build`, diretório de saída `dist`.
3. Cadastre `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY` e `VITE_N8N_WEBHOOK_URL` em *Environment Variables*.
4. `vercel.json` reescreve todas as rotas para `index.html` (necessário para a SPA).
5. Em Supabase → Authentication → URL Configuration, cadastre o domínio de produção.

12.2 Banco e funções (Supabase)

```
supabase link --project-ref <id-do-projeto>
supabase db push                                  # aplica migrações pendentes
supabase functions deploy fetch-aplv-news
supabase secrets set NEWS_API_KEY=<chave>
```

12.3 API FastAPI

Execute com `uvicorn app.main:app --host 0.0.0.0 --port 8000` em um serviço de contêiner (Render, Railway, Fly.io, Cloud Run). Ajuste o CORS para o domínio de produção e defina as variáveis de ambiente da seção 5.2.

12.4 Checklist de publicação

- [ ] `npm run lint` e `npm run test` sem erros
- [ ] `npm run build` concluído
- [ ] Migrações aplicadas e tipos regenerados
- [ ] Variáveis de ambiente conferidas no Vercel
- [ ] Documentação atualizada e PDFs regerados (`npm run docs:pdf`)

13. Manutenção da documentação

A documentação vive junto ao código, em `docs/`, e deve ser atualizada **no mesmo commit** de qualquer mudança que a afete.

Mudança no sistema	Documento(s) a atualizar
Nova tela, fluxo ou texto visível ao usuário	Manual do Usuário
Nova tabela, coluna, função ou política	Documentação Técnica (seção 6) e Arquitetura (seção 8)
Novo serviço, endpoint ou integração	Documentação Técnica (seções 7–8) e Arquitetura (seções 6–7)
Mudança na regra de classificação ou alternativas	Arquitetura (seção 9) e Documentação Técnica (seção 7)
Correção de item de dívida técnica	Arquitetura (seção 13)

Passos:

1. Edite o(s) arquivo(s) `.md` em `docs/`.
2. Atualize o campo `versao` e `atualizado` no cabeçalho e adicione uma linha ao **Histórico de revisões**. Use versionamento semântico: correção de texto → `x.y.Z`; seção ou funcionalidade nova → `x.Y.0`; reestruturação → `X.0.0`.
3. Gere os PDFs: `npm run docs:pdf`.
4. Faça commit dos `.md` e dos `.pdf` juntos.

O gerador (`scripts/build-docs.mjs`) monta capa, sumário e diagramas Mermaid e imprime com o Chrome/Edge em modo *headless*. Se o navegador estiver em local não padrão, defina `CHROME_PATH`.

14. Histórico de revisões

Versão	Data	Alterações
1.0.0	12/11/2026	Primeira versão completa da documentação técnica.
1.0.1	12/11/2026	`create-admin-user` restrita a administradores (seção 8.2); tratamento de erro do scanner (seções 8.4 e 9).
1.1.0	12/11/2026	Paginação de 30 itens no módulo Remédios: `getMedicationsPage`, `MEDS_PAGE_SIZE`, busca em lotes em `getMedicationsByRisk` e componente `PaginationBar` (seções 7.1 e 9).
1.2.0	12/11/2026	LGPD: tabela `user_consents`, Edge Function `delete-account`, `services/privacy.ts`, `services/auth.ts`, `config/legal.ts`; campos opcionais no perfil; fonte `font-display` (seções 6, 7, 8, 9 e 10).
1.3.0	12/11/2026	Menu lateral redesenhado: `services/profile.ts`, `lib/names.ts` e testes (seções 7.5, 9 e 11).
1.4.0	12/11/2026	`PageHeader`, `lib/theme.ts`, `displayName` / `firstName`; perfil sem estatísticas fictícias; scanner aceita foto da galeria (seções 7.5 e 9).
1.5.0	12/11/2026	`services/labelAnalysis.ts`: leitura local do rótulo com Tesseract.js como reserva do n8n, regras de leite testadas (seções 7.5.2 e 8.5).
1.6.0	12/11/2026	Scanner: foto enviada ao lado do resultado, com visualização ampliada (seção 9).
1.7.0	12/11/2026	Leitura local: preparação da imagem (cinza, contraste, endireitamento automático), duas leituras e tolerância a erros de letra; novos resultados de teste (seção 7.5.2).
1.8.0	12/11/2026	Diagnóstico das colunas ausentes no banco em produção e script de correção (seção 6.5.1); cadastro grava perfil essencial e complementar separadamente.
1.9.0	12/11/2026	Alergias por tipo: colunas `food_allergies` / `drug_allergies` e migração (6.2, 6.5); `lib/allergens.ts`, `lib/medicationRisk.ts` (regra única `assessMedication`), `getAllergySelection` / `saveAllergySelection`, `useUserAllergies`, `AllergyPicker`, `AllergySummary`; `getMedicationsByRisk`, `getSafeAlternatives`, `classifyLabelText` / `analyzeLabel` passam a usar as alergias do perfil (7.1, 7.2.1, 7.5, 7.5.2).
1.10.0	12/11/2026	Página de mensagens do administrador: colunas de status e políticas em `suggestions` / `feedback`, função `admin_list_messages`, migração e script manual (6.2, 6.4, 6.5); `services/adminMessages.ts`, `roleLoading` (7.5.3); `AdminMessagesPage` e item no menu (9).
1.11.0	12/11/2026	Bulas no Storage: colunas `bula_*`, `bula_versoes`, bucket e funções (6.3, 6.4, 6.5); `services/bulas.ts`, script `bulas:sync` (4.3, 7.5.4); `BulaViewer` (9); `bula_reanalisar` → Atenção.